

CSCI 2406 | Computer Organization & Architecture | TOPIC 6

Data Transfer and Addressing

Load/store instructions, addressing modes,
memory alignment, arrays, and pointers

Dr. J. Burns

Topic 6 Outline

Topic Objective

Explain how AArch64 transfers values between registers and memory and how an instruction forms the effective address for a memory access.

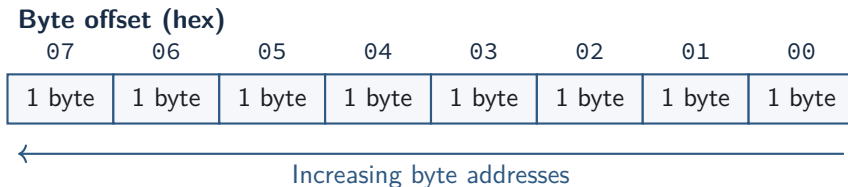
- **Section 1:** Alignment and transfer widths
- **Section 2:** AArch64 addressing modes
- **Section 3:** Pair transfers
- **Section 4:** Summary and self-test

Section 1

Alignment and Transfer Widths

Memory Is Byte Addressed

- Each memory address identifies one byte.
- Multi-byte values occupy consecutive addresses.
- The instruction determines how many bytes are transferred.



Example: One 64-Bit Value

Starting address: 1000 (hexadecimal). The rightmost byte is at 1000; the next byte to its left is at 1001; the leftmost byte is at 1007. Together, the eight bytes occupy 64 bits.

Natural Alignment

Natural Alignment Rule

An n -byte object is naturally aligned when its starting address is a multiple of n .

- 2-byte halfword: address divisible by 2.
- 4-byte word: address divisible by 4.
- 8-byte doubleword: address divisible by 8.

AArch64 Qualification

Many ordinary AArch64 loads and stores to normal memory permit unaligned addresses, but natural alignment is preferred. Misaligned accesses may be slower or cross memory-system boundaries, and some access classes have stricter requirements.

Natural Alignment Boundaries

Byte offset (hex)	07	06	05	04	03	02	01	00
8-bit (1 byte)								
16-bit (2 bytes)	2 bytes		2 bytes		2 bytes		2 bytes	
32-bit (4 bytes)	4 bytes				4 bytes			
64-bit (8 bytes)	8 bytes							

Natural Alignment for Loads and Stores

Let A be the effective byte address. For an n -byte transfer, natural alignment means $A \bmod n = 0$.

ldrb/strb: $A \bmod 1 = 0$ ldrrh/strrh: $A \bmod 2 = 0$

ldr/str (32-bit): $A \bmod 4 = 0$ ldr/str (64-bit): $A \bmod 8 = 0$

Offset 00 starts at 1000 (hex); each group begins at its right edge. These are natural alignment conditions; ordinary loads/stores to normal memory may permit unaligned addresses.

Consequences of Unaligned Access

Example

An 8-byte access at address 1003 (hex) is not naturally aligned: $A \bmod 8 \neq 0$.

- **Correct access may still be possible:** ordinary AArch64 loads/stores to normal memory often support unaligned addresses.
- **Lower performance:** crossing a cache-line boundary may require extra work.
- **Possible fault:** alignment checking, certain instructions, or device memory can prohibit unaligned access.
- **Atomicity may not be guaranteed:** an unaligned access may lack the guarantees of a naturally aligned access.

Why Prefer Natural Alignment?

An unaligned access does not automatically produce the wrong value. Natural alignment avoids these complications.

Load/Store Transfer Widths

Load	Store	Width	Load Result
ldrb	strb	1 byte	Zero-extended to 32 bits
ldrh	strh	2 bytes	Zero-extended to 32 bits
ldr (32-bit)	str (32-bit)	4 bytes	32-bit value; upper 32 bits cleared
ldr (64-bit)	str (64-bit)	8 bytes	Full 64-bit value

- A load into a 32-bit register clears the upper half of its 64-bit register.
- Stores write the specified number of bytes from the low end of the source register.
- The instruction determines transfer width; the memory address carries no type.

Signed Loads Preserve Negative Values

Sign Extension

A signed load copies the sign bit of the narrow source into the new upper bits so the two's-complement value is preserved.

Instruction	Source Width	Result
<code>ldrsh x1, [x0]</code>	8 bits	Sign-extended to 64 bits
<code>ldrsh x1, [x0]</code>	16 bits	Sign-extended to 64 bits
<code>ldrsw x1, [x0]</code>	32 bits	Sign-extended to 64 bits

Example

If memory contains the byte `0xff`, `ldrb` interprets the transferred bits as 255 after zero extension, while `ldrsh x1, [x0]` produces the 64-bit representation of `-1`.

Load Width and Extension Example

Assume $x0 = 0x2000$ and memory at that address contains the byte $0x80$.

- **Unsigned byte load:** `ldrb w1, [x0]`
 - Reads one byte and fills the remaining bits with zeros.
 - Result: $w1 = 0x00000080$, decimal value 128.
 - Writing $w1$ also clears the upper 32 bits of $x1$.
- **Signed byte load:** `ldrsb x1, [x0]`
 - Reads one byte and copies its sign bit into all higher bits.
 - Result: $x1 = 0xffffffffffff80$, decimal value -128 .

Section 2

AArch64 Addressing Modes

Three Addressing Types

Pointer and Base Register

A **pointer** holds a memory address. The **base register** holds that pointer. Throughout these examples, x0 holds the C pointer ptr; loaded values go into x1 (or w1 for a narrow load).

- ❶ **Offset:** access an address relative to the pointer, leaving the pointer unchanged.
- ❷ **Pre-index:** adjust the pointer first, then access memory at the adjusted address.
- ❸ **Post-index:** access memory at the current pointer, then adjust the pointer.

The Effective Address

Definition

The **effective address** is the memory address computed for a particular load or store instruction.

- A base register holds the pointer that supplies the **starting address**.
- An optional immediate or register offset can **modify** that address.
- Some addressing forms also **write back** an updated address to the base register.

Base Register

AArch64 memory operands normally use a 64-bit general-purpose register or sp as the base. **A w register is not used as the base address register.**

1. Offset: Base Register Only

Syntax

```
ldr x1, [x0]
```

- Effective address = x0.
- x0 is unchanged.
- This corresponds closely to dereferencing a pointer already holding the required address.

Equivalent C: ptr Points to 64-Bit Elements

```
uint64_t *ptr;  
value = *ptr;
```

This is offset addressing with a zero offset: read the current element and leave ptr unchanged.

1. Offset: Immediate Offset

Example

```
ldr x1, [x0, #16]
```

- Effective address = $x0 + 16$.
- The base register $x0$ is unchanged.
- Useful for structure fields and fixed-position array elements.

Equivalent C

```
value = *(ptr + 2);
```

Read the element 16 bytes beyond `ptr`; `ptr` is unchanged.

1. Offset: Register Offset

1. Register Holds the Byte Offset

AArch64: `ldr x1, [x0, x2]`

C: `value = *(ptr + offset);`

C counts offset in elements. x2 holds `offset * 8`, the corresponding byte offset.

2. Element Index Converted to a Byte Displacement

AArch64: `ldr x1, [x0, x2, lsl #3]`

C: `value = *(ptr + i);`

x2 holds index i; `lsl #3` multiplies it by 8.

- In both examples, x0 holds ptr, pointing to 64-bit elements.
- The loaded value goes into x1; x0 is unchanged.

2. Pre-Indexed Addressing

Assembly

```
ldr x1, [x0, #8]!
```

Equivalent C: Advance, Then Read

```
ptr++;
```

```
value = *ptr;
```

Or, in one expression: `value = *++ptr;`

- Access address: original `x0 + 8` (the next element).
- Base after the instruction: original `x0 + 8`.
- The exclamation mark indicates writeback; A64 pre-index uses an immediate offset.

3. Post-Indexed Addressing

Assembly

```
ldr x1, [x0], #8
```

Equivalent C: Read, Then Advance

```
value = *ptr;
```

```
ptr++;
```

Or, in one expression: `value = *ptr++;`

- Access address: original `x0` (the current element).
- Base after the instruction: original `x0` + 8.
- The immediate outside the brackets specifies the pointer update.

Addressing Mode Comparison

Form	Example	Access Address	Base After
Offset (zero)	<code>ldr x1, [x0]</code>	<code>x0</code>	unchanged
Immediate offset	<code>ldr x1, [x0, #16]</code>	<code>x0 + 16</code>	unchanged
Register offset	<code>ldr x1, [x0, x2]</code>	<code>x0 + x2</code>	unchanged
Pre-index	<code>ldr x1, [x0, #8]!</code>	<code>x0 + 8</code>	<code>x0 + 8</code>
Post-index	<code>ldr x1, [x0], #8</code>	<code>x0</code>	<code>x0 + 8</code>

Key Distinction

Offset addressing computes an access address without changing the base register; pre-indexed and post-indexed forms also write an updated address back to the base.

Byte Access: Advance by One Byte

C: Read One Byte, Then Advance

```
uint8_t *ptr;  
value = *ptr++;
```

AArch64: x0 Holds ptr

```
ldrb w1, [x0], #1
```

- Read one byte and zero-extend it into w1.
- Advance by 1 byte for a byte array, compared with 8 bytes for a 64-bit array.
- Match both the load width and pointer increment to the element size.

Section 3

Pair Transfers

Load and Store Pair: `ldp` and `stp`

Store Pair

```
stp x1, x2, [x0]
```

- Stores two register values.
- The memory locations are adjacent.

Load Pair

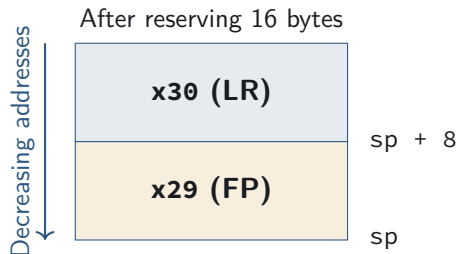
```
ldp x1, x2, [x0]
```

- Loads two register values.
- The memory locations are adjacent.

Important Qualification

A pair transfer is one architectural instruction, but it is not guaranteed to be atomic or to execute as one physical memory-bus transaction.

Pair Transfers: Order in Memory



Reserve Space, Then Store

```
sub sp, sp, #16
stp x29, x30, [sp]
```

Store FP, then LR in the adjacent slot.

Load, Then Release Space

```
ldp x29, x30, [sp]
add sp, sp, #16
```

Restore the pair, then restore the original sp.

Pointer Adjustment and Stack Uses

FP: frame pointer (x29); **LR**: link register (x30), holding the return address. Only sub/add change sp here. Stack frames and saving these registers will be developed in the stack lecture.

Section 4

Summary and Self-Test

Topic 6 Summary

- AArch64 uses explicit load/store instructions to move values between registers and memory.
- The instruction determines transfer width and whether a narrow loaded value is zero-extended or sign-extended.
- The base register holds a pointer. Offset addressing leaves it unchanged; pre-index and post-index addressing update it before or after the access, respectively.
- Scaled register offsets map naturally to array indexing; post-indexing maps naturally to pointer traversal.
- Natural alignment is preferred, although many ordinary unaligned accesses to normal memory are permitted.
- `ldp` and `stp` transfer adjacent register pairs without implying atomicity or a particular physical bus transaction.

Self-Test 1/2

- 1 If C pointer `ptr` is held in base register `x0`, what C statement corresponds to `ldr x1, [x0]`? What do the brackets mean?
- 2 How many bytes are read by `ldrb`, `ldrh`, `ldr w1, [...]`, and `ldr x1, [...]`?
- 3 Memory contains byte `0x80`. What decimal values result from `ldrb w1, [x0]` and `ldrsb x1, [x0]`? Explain the extension used.
- 4 Name the three addressing types. Which leave the base pointer unchanged, and which update it?
- 5 For `ldr x1, [x0, #16]`, give the effective address and equivalent C statement, assuming `ptr` points to 64-bit elements.

Self-Test 2/2

- 6 Translate `value = *(ptr + i);` into a load using `x0` for `ptr` and `x2` for `i`. Assume 64-bit elements. Why is a shift of 3 needed?
- 7 Compare `ldr x1, [x0, #8]!` and `ldr x1, [x0], #8`. Give the access address, final base value, and equivalent C statements for each.
- 8 For a byte pointer, translate `value = *ptr++;` using `x0` for `ptr`. Why is the increment 1 rather than 8?
- 9 What does natural alignment mean for an 8-byte object? Must every ordinary AArch64 load use a naturally aligned address?
- 10 What do `ldp` and `stp` transfer? Do they guarantee an atomic transfer?

Branching and Iteration

Topic 7 develops the comparison, branching, and control-flow instructions used to implement decisions and repetition.

- Comparison instructions
- Conditional and unconditional branches
- Loops and selection structures
- Translation of high-level control flow